New Relic APM — Day 1 Lab Guide

Hands-on labs for *Deep dive into transactions, traces and services* · easyTravel on Linux · New Relic Java agent

How to use this guide

This guide holds the four hands-on labs for Day 1. Each lab matches a module of the Day 1 Study Guide: read the module, then do its lab. The study guide is a separate document: *New Relic APM — Day 1 Study Guide*.

Lab	Title	Module	Time
1.0	Instrument easyTravel with the New Relic Java agent	Set-up	30 min
1.1	Isolate the root cause of a slow checkout transaction	Module 1	45 min
1.2	Follow a failed request end-to-end and pinpoint the failing hop	Module 2	50 min
1.3	Correlate a post-deployment latency spike with traces, logs and the change record	Module 3	50 min

Lab environment

Item	Requirement
Linux VM	Ubuntu 22.04 or 24.04 LTS, 4 vCPU, 8 GB RAM, 30 GB disk, outbound HTTPS to New Relic
Software	Docker Engine with the Compose plugin, `git`, `curl`, `unzip`, `jq` (installed in Lab 1.0)
Application	**easyTravel** — Dynatrace's open-source demo travel agency, run from its official Docker images
New Relic	An account (the free tier is enough), its **license key** (type *INGEST – LICENSE*), a **user key** (for the CLI in Lab 1.3) and the **account ID**
Browser	Any modern browser on your workstation, to reach easyTravel on `http://<VM-IP>/` and New Relic at `one.newrelic.com` (or `one.eu.newrelic.com`)

easyTravel runs as these containers:

Container	What it is	Instrumented
`www`	nginx, the entry point on port 80	no
`frontend`	Customer web site (Java, Tomcat) → **easyTravel-frontend**	Java agent
`backend`	Business backend web services (Java, Tomcat), port 8091 on the VM → **easyTravel-backend**	Java agent
`mongodb`	The database	seen as a datastore from the backend
`headless-customer-loadgen`	Headless Chrome that searches, logs in and books journeys all day	no

The `angular-frontend` container also starts (nginx needs it) but it is not used in the labs.

Conventions

- `$` lines are typed in a terminal on the lab VM as the user `trainee`, from `~/nr-lab` unless a step says otherwise.
- **Expected output** pictures show what a terminal should print. **Expected view** pictures are redrawn New Relic screens: the layout, names and numbers on your screen will differ — look for the *shape* they show (a step, a spike, a repeated call, a red span).
- New Relic changes its UI often. If a menu path does not match, use the search bar at the top of New Relic, or the NRQL query given in the same step — the queries always work.
- Secrets: the license key and the user key are **lab secrets**. They live only in `~/nr-lab/.env` (mode 600). Never paste them into a ticket or chat, and never commit `.env` to Git.

Problem patterns

easyTravel has built-in *problem patterns* — plugins that you switch on and off through the backend's configuration service. You will use two of them, plus one failure you cause by hand:

Pattern	What it does	Used in
`DBSpamming`	Unnecessary database access in the backend when a booking is stored	Lab 1.1
`DBSpammingAuthWithAppDeployment`	Simulates an upgrade of the application, then spams the database when a user logs in	Lab 1.3
(no plugin — you stop MongoDB)	Every backend call that needs the database fails	Lab 1.2

```
# switch a pattern on (register once, then enable)
curl -s "http://localhost:8091/services/ConfigurationService/registerPlugins?pluginData=
<Pattern>"
curl -s "http://localhost:8091/services/ConfigurationService/setPluginEnabled?name=
<Pattern>&enabled=true"
# switch it off
curl -s "http://localhost:8091/services/ConfigurationService/setPluginEnabled?name=
<Pattern>&enabled=false"
```

In Lab 1.0 you wrap these calls in a small shell function, `pattern on|off <Pattern>`.

Lab 1.0 — Instrument easyTravel with the New Relic Java agent

Goal: run easyTravel on the VM with the New Relic Java agent in the frontend and backend Tomcats, with distributed tracing and logs in context switched on, and see both services report to New Relic.

Time: 30 minutes.

Step 1 — Check the VM and install the tools

```
sudo apt-get update && sudo apt-get install -y git curl unzip jq
# Docker Engine + Compose plugin (skip if docker compose version already works)
curl -fsSL https://get.docker.com | sudo sh
sudo usermod -aG docker $USER
# log out and back in (or run: newgrp docker) so the group change applies, then:
docker version --format 'Docker {{.Server.Version}}'
docker compose version
nproc; free -h | head -2
```

Lab 1.0 · Step 1 · Docker, Compose and VM size — EXPECTED OUTPUT

```
trainee@nr-lab:~/nr-lab$ docker version --format 'Docker {{.Server.Version}}'
Docker 28.5.1
trainee@nr-lab:~/nr-lab$ docker compose version
Docker Compose version v2.40.0
trainee@nr-lab:~/nr-lab$ nproc; free -h | head -2
4
               total        used        free      shared  buff/cache   available
Mem:           7.8Gi       598Mi       6.5Gi       1.2Mi       0.9Gi       7.2Gi
4 vCPU and ~8 GB: enough for easyTravel and the headless load generator
```

Expected output — Docker and Compose installed; 4 CPUs and 8 GB of memory available

Step 2 — Get easyTravel and the Java agent

```
mkdir -p ~/nr-lab && cd ~/nr-lab
git clone --depth 1 https://github.com/Dynatrace/easyTravel-Docker.git easytravel
curl -sSLO https://download.newrelic.com/newrelic/java-agent/newrelic-agent/current/newrelic-java.zip
unzip -q newrelic-java.zip            # creates ./newrelic/ with newrelic.jar and a sample newrelic.yml
mkdir -p agent/frontend agent/backend
cp newrelic/newrelic.jar agent/frontend/
cp newrelic/newrelic.jar agent/backend/
unzip -p agent/backend/newrelic.jar META-INF/MANIFEST.MF | grep Implementation-Version
```

Each tier gets its own agent folder, so that each has its own `newrelic.yml` and its own agent log (`agent/<tier>/logs/newrelic_agent.log`).

Step 3 — Store the keys

```
cat > ~/nr-lab/.env <<'EOF'
NEW_RELIC_LICENSE_KEY=<YOUR_LICENSE_KEY>
NEW_RELIC_API_KEY=<YOUR_USER_KEY>
NEW_RELIC_ACCOUNT_ID=<YOUR_ACCOUNT_ID>
NEW_RELIC_REGION=US
```

```
EOF
chmod 600 ~/nr-lab/.env
```

Replace the three placeholders with your own values (use `EU` as the region if your account is in the EU data center). Docker Compose reads the file through `--env-file` in Step 5, so there is only one copy of the keys.

Step 4 — Write the agent configuration

One `newrelic.yml` per tier. Only the settings that matter today are written out; everything else keeps the agent default. The license key is **not** in the file — the agent reads it from the `NEW_RELIC_LICENSE_KEY` environment variable.

```
for tier in frontend backend; do
cat > ~/nr-lab/agent/$tier/newrelic.yml <<EOF
common: &default_settings
  app_name: easyTravel-$tier
  log_level: info
  # Module 2: distributed tracing (on by default since agent 7.4 - written out to be
explicit)
  distributed_tracing:
    enabled: true
    exclude_newrelic_header: false
  span_events:
    enabled: true
    max_samples_stored: 2000
  # Module 1: transaction traces and slow SQL/datastore details
  transaction_tracer:
    enabled: true
    transaction_threshold: apdex_f
    record_sql: obfuscated
    explain_threshold: 0.5
  # Module 3: logs in context
  application_logging:
    enabled: true
    forwarding:
      enabled: true
      max_samples_stored: 10000
    metrics:
      enabled: true
production:
  <<: *default_settings
EOF
done
grep -n "app_name\|enabled: true" ~/nr-lab/agent/backend/newrelic.yml
```

Step 5 — Add the agent to the Tomcat containers

The easyTravel images start Tomcat with `catalina.sh run`, so the agent is added with `-javaagent` in `CATALINA_OPTS`. Instead of editing the project's `docker-compose.yml`, write an override file next to it:

```
cat > ~/nr-lab/easytravel/docker-compose.newrelic.yml <<'EOF'
services:
  backend:
    environment:
      CATALINA_OPTS: "-javaagent:/opt/newrelic/newrelic.jar -Dconfig.apmServerDefault=APM -
Xmx300m"
      NEW_RELIC_LICENSE_KEY: ${NEW_RELIC_LICENSE_KEY}
```

```
    volumes:
      - ../agent/backend:/opt/newrelic
  frontend:
    environment:
      CATALINA_OPTS: "-javaagent:/opt/newrelic/newrelic.jar -Dconfig.apmServerDefault=APM -
Xmx525m"
      NEW_RELIC_LICENSE_KEY: ${NEW_RELIC_LICENSE_KEY}
    volumes:
      - ../agent/frontend:/opt/newrelic
EOF
```

Make a short alias (`etdc` = *easyTravel docker compose*) so every later step uses both files and the keys in `.env`:

```
echo "alias etdc='docker compose -f ~/nr-lab/easytravel/docker-compose.yml -f ~/nr-
lab/easytravel/docker-compose.newrelic.yml --env-file ~/nr-lab/.env'" >> ~/.bashrc
source ~/.bashrc
```

Add a helper that switches easyTravel problem patterns on and off and prints the HTTP status:

```
cat >> ~/.bashrc <<'RC'
pattern() {   # usage: pattern on|off <PatternName>
  local B=http://localhost:8091/services/ConfigurationService s=true
  [ "$1" = off ] && s=false
  [ "$1" = on ] && curl -s -o /dev/null "$B/registerPlugins?pluginData=$2"
  curl -s -o /dev/null -w "$2 enabled=$s -> HTTP %{http_code}\n" "$B/setPluginEnabled?
name=$2&enabled=$s"
}
RC
source ~/.bashrc
```

Step 6 — Start easyTravel

```
etdc pull mongodb backend frontend www headless-customer-loadgen
etdc up -d mongodb backend frontend www headless-customer-loadgen
sleep 90; etdc ps --format "table {{.Name}}\t{{.Status}}\t{{.Ports}}"
```

Lab 1.0 · Step 6 · Start easyTravel — EXPECTED OUTPUT

```
trainee@nr-lab:~/nr-lab$ etdc up -d mongodb backend frontend www headless-customer-loadgen
[+] Running 7/7
 ✔ Network easytravel_default           Created
 ✔ Container mongodb                    Started
 ✔ Container backend                    Started
 ✔ Container frontend                   Started
 ✔ Container angular-frontend           Started
 ✔ Container www                        Started
 ✔ Container headless-customer-loadgen  Started
trainee@nr-lab:~/nr-lab$ sleep 90; etdc ps --format "table {{.Name}}\t{{.Status}}\t{{.Ports}}"
NAME                       STATUS              PORTS
angular-frontend           Up About a minute   0.0.0.0:32771->8080/tcp
backend                    Up About a minute   0.0.0.0:8091->8080/tcp
frontend                   Up About a minute   0.0.0.0:32770->8080/tcp
headless-customer-loadgen  Up About a minute
mongodb                    Up About a minute   0.0.0.0:32768->27017/tcp,
    0.0.0.0:32769->28017/tcp
www                        Up About a minute   0.0.0.0:80->80/tcp, 0.0.0.0:8080->8080/tcp,
    0.0.0.0:9079->9079/tcp
```

Expected output — easyTravel containers running; backend on 8091 and nginx on 80

Open `http://<VM-IP>/` in your browser: the easyTravel home page appears. Log in with one of the demo users shown on the login page (for example `maria` / `maria`), search for a journey and book it once — that is the **checkout** flow you will troubleshoot in Lab 1.1.

Step 7 — Check that both agents connected

```
for t in frontend backend; do
  echo "== $t"; grep -E "Agent v[0-9]|Reporting to|ERROR" ~/nr-
lab/agent/$t/logs/newrelic_agent.log | tail -3
done
etdc exec backend sh -c 'echo $CATALINA_OPTS'
```

Lab 1.0 · Step 7 · Both agents connected — EXPECTED OUTPUT

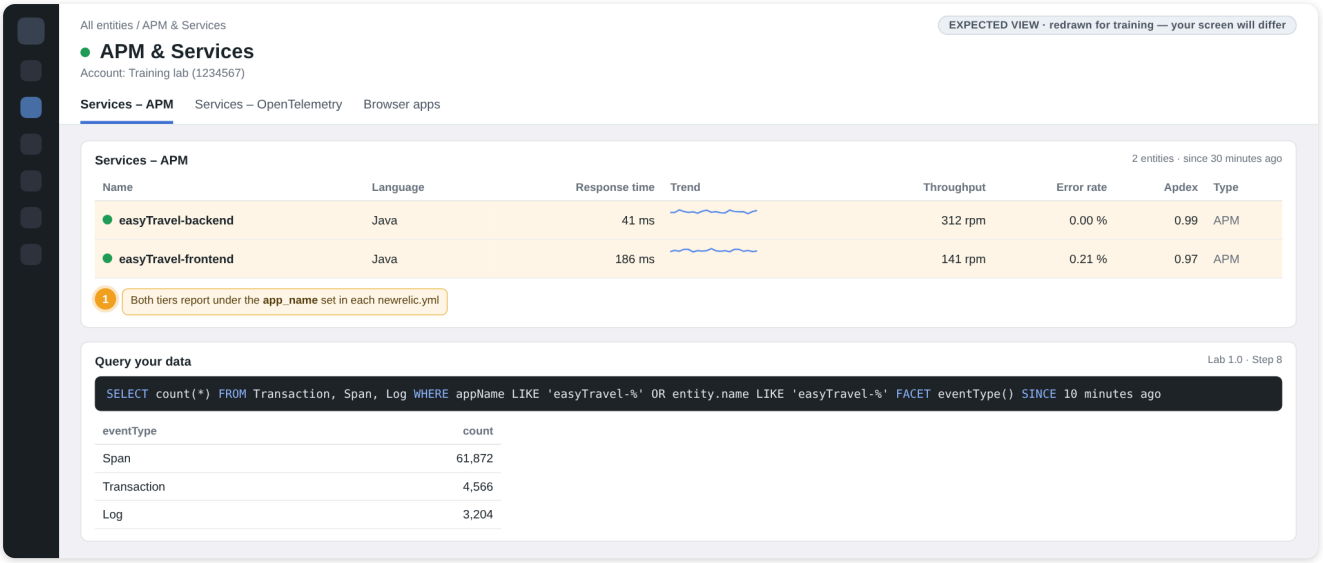
```
trainee@nr-lab:~/nr-lab$ for t in frontend backend; do echo "== $t"; grep -E "Agent
                         v[0-9]|Reporting to|ERROR"
                         ~/nr-lab/agent/$t/logs/newrelic_agent.log | tail -3; done
== frontend
2026-10-26T09:41:07,214+0000 [1 1] com.newrelic INFO: New Relic Agent v8.23.0 is initializing...
2026-10-26T09:41:33,902+0000 [1 58] com.newrelic INFO: Reporting to:
    https://rpm.newrelic.com/accounts/1234567/applications/601234561
== backend
2026-10-26T09:41:06,877+0000 [1 1] com.newrelic INFO: New Relic Agent v8.23.0 is initializing...
2026-10-26T09:41:29,410+0000 [1 61] com.newrelic INFO: Reporting to:
    https://rpm.newrelic.com/accounts/1234567/applications/601234568
trainee@nr-lab:~/nr-lab$ etdc exec backend sh -c 'echo $CATALINA_OPTS'
-javaagent:/opt/newrelic/newrelic.jar -Dconfig.apmServerDefault=APM -Xmx300m
no ERROR lines: both agents connected. The account and application IDs will differ.
```

Expected output — both agents initialised and reporting to New Relic; the backend runs with -javaagent

If a tier shows nothing, see *Troubleshooting* in the Appendix.

Step 8 — Find the services in New Relic

In New Relic, open **APM & Services**. After two or three minutes of traffic, two services appear: **easyTravel-frontend** and **easyTravel-backend**, both with throughput, response time and error rate.



Expected view — APM & Services lists easyTravel-frontend and easyTravel-backend

Confirm from NRQL (*Query your data*) that all three data types arrive:

```
SELECT count(*) FROM Transaction, Span, Log
WHERE appName LIKE 'easyTravel-%' OR entity.name LIKE 'easyTravel-%'
FACET eventType() SINCE 10 minutes ago
```

Check your work

- ☐ Both services are listed in APM & Services and show throughput.
- ☐ `Transaction` and `Span` both return rows. (`Log` rows are checked again in Lab 1.3.)
- ☐ `.env` is mode 600 and holds the only copy of the keys.

Lab 1.1 — Isolate the root cause of a slow checkout transaction

Goal: a release has made checkout slow. Use the APM Summary, the transactions breakdown table and a transaction trace to find *where* the time goes and *why* — and prove it with NRQL.

Time: 45 minutes. **Module:** 1.

Step 1 — Record a baseline

Let the load generator run for at least 10 minutes with no problem pattern. Open **easyTravel-backend** → **Summary** and note the web transaction time and throughput. Run this query and keep the result — it is your *before* number:

```
SELECT average(duration), average(databaseCallCount), count(*) FROM Transaction
WHERE appName = 'easyTravel-backend' FACET request.uri SINCE 10 minutes ago
```

Step 2 — Break checkout

```
pattern on DBSpamming
date +%H:%M
```

Lab 1.1 · Step 2 · Switch on DBSpamming — EXPECTED OUTPUT

```
trainee@nr-lab:~/nr-lab$ pattern on DBSpamming
DBSpamming enabled=true -> HTTP 200
trainee@nr-lab:~/nr-lab$ date +%H:%M
10:20
write the time down: the Summary chart should step up right after it
```

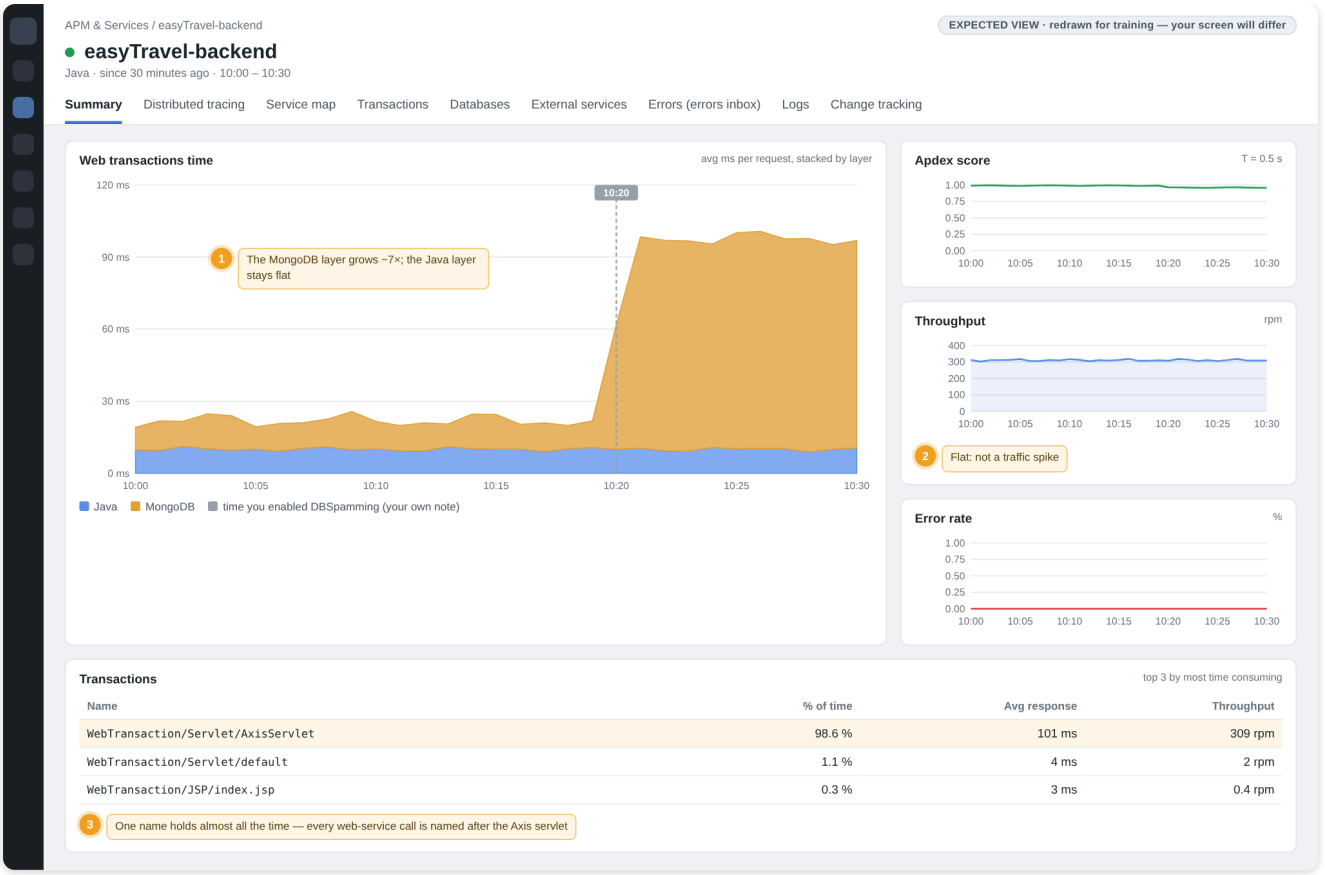
Expected output — DBSpamming registered and enabled on the backend

Write down the time. Make two or three bookings in the browser yourself as well — you will feel the slower *Book* button. Wait 5 minutes.

Step 3 — Read the Summary

Open **easyTravel-backend** → **Summary** for the last 30 minutes. Look at:

- **Web transactions time**: the stacked chart steps up at the time you wrote down, and the **MongoDB** layer grows far more than the Java layer.
- **Throughput**: roughly flat — this is not a traffic spike.
- **Error rate**: flat — the requests are slow, not failing.
- **Transactions** table on the Summary: the slowest-by-time transaction.



Expected view — backend Summary: web transaction time steps up and the MongoDB layer dominates

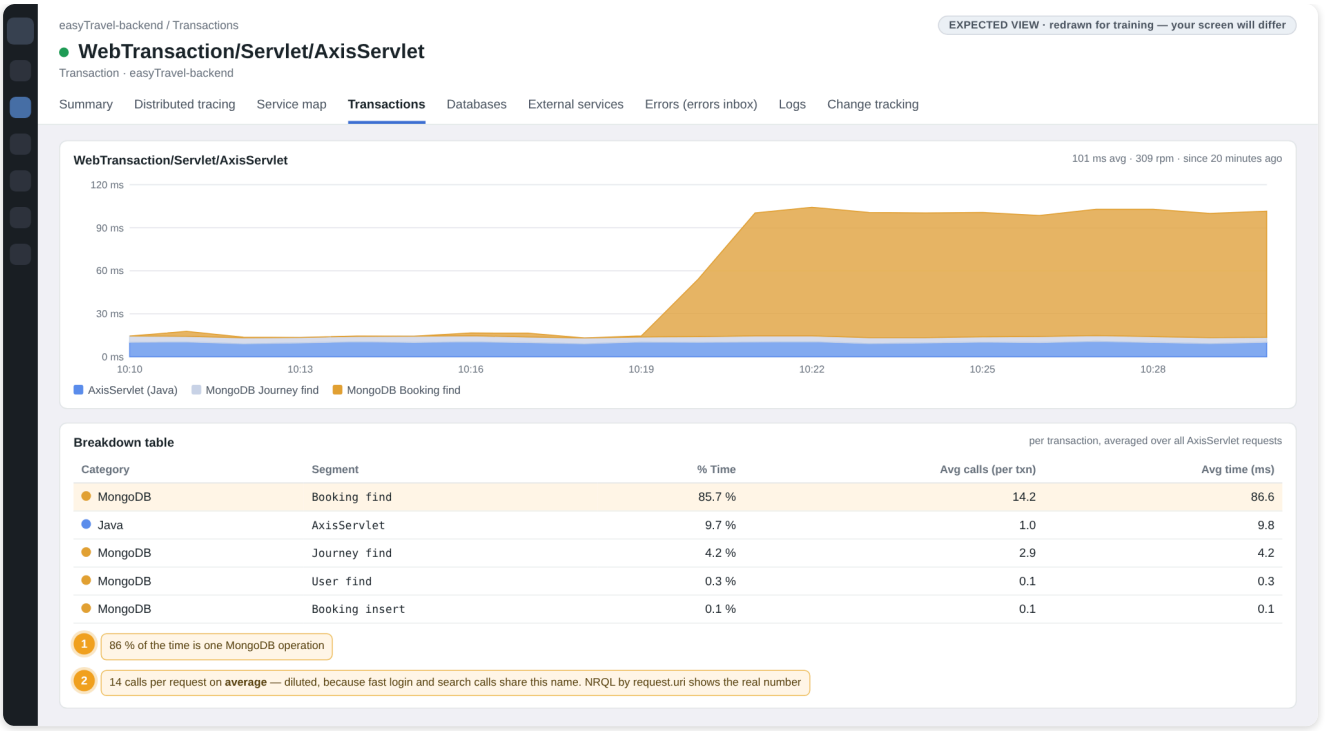
Question 1. Which layer of the stacked chart grew? What does that rule *out*?

Step 4 — Find the transaction and the segment

Open **Transactions**, sort by **Most time consuming**, and open the top one.

> **Naming check.** easyTravel's backend runs Apache Axis2 web services behind a single servlet. On many agent versions every backend call is therefore named after that servlet (for example `WebTransaction/Servlet/AxisServlet`) instead of after the operation. That is the *naming issue that hides real problems* from Module 1: a slow booking call is averaged together with fast login and search calls. Use `request.uri` (below) to separate them. If your agent names each operation separately, simply open the booking one.

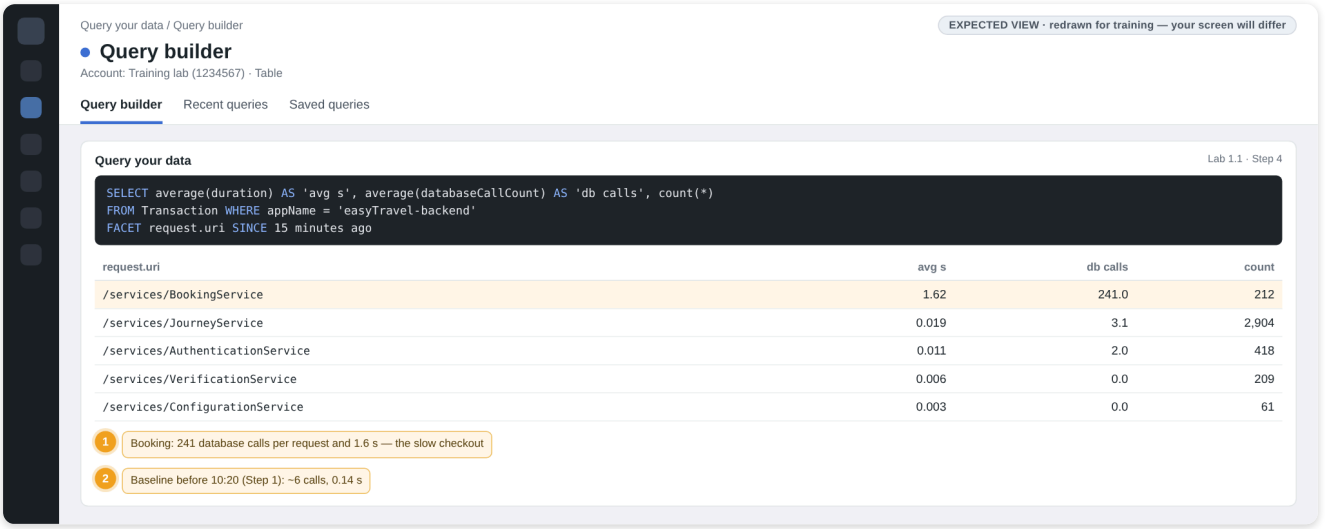
Read the **breakdown table** under the chart. Note for the MongoDB rows: *% time*, *avg calls per transaction* and *avg time per call*.



Expected view — breakdown table: one MongoDB operation takes 86% of the time; its call count is diluted by the shared name

Now separate the operations with NRQL:

```
SELECT average(duration) AS 'avg s', average(databaseCallCount) AS 'db calls', count(*)
FROM Transaction WHERE appName = 'easyTravel-backend'
FACET request.uri SINCE 15 minutes ago
```



Expected view — NRQL: the booking service makes 241 database calls per request; the others make 0 to 3

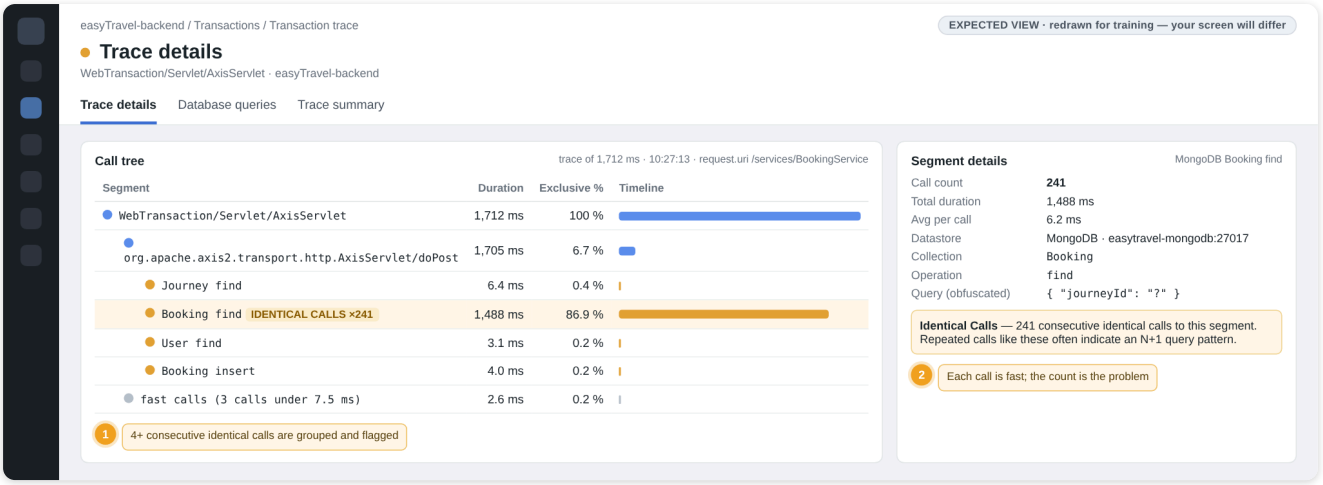
Question 2. The MongoDB `find` average is a few milliseconds per call. Is the database slow?

Step 5 — Open a transaction trace

On the transaction page, scroll to **Transaction traces** (or *Slow transaction traces*) and open the slowest trace whose URI is the booking service. In **Trace details**:

1. Switch to the **call tree** (waterfall / timeline also work).
2. Find the segment with the **Identical Calls** badge — four or more consecutive identical calls.

3. Expand it: the same MongoDB query runs once per item, each call fast, all together most of the transaction.
4. Open the segment's **Details**/**Database** tab: the obfuscated query, the collection and the call count.



Expected view — trace call tree: 241 identical MongoDB find calls flagged in one booking request

This is an **N+1 pattern**: one query to fetch a list, then one more query per item. Tuning the single query (an index, a faster disk) would barely help. The fix is in code: fetch everything in one query (or cache it).

Step 6 — Prove it before and after

```
SELECT average(databaseCallCount), average(databaseDuration), average(duration)
FROM Transaction WHERE appName = 'easyTravel-backend'
AND request.uri LIKE '%Booking%'
SINCE 15 minutes ago COMPARE WITH 45 minutes ago
```

Then switch the pattern off and confirm recovery after 5 minutes:

```
pattern off DBSpamming
```

Lab 1.1 · Step 6 · Switch DBSpamming off — EXPECTED OUTPUT

```
trainee@nr-lab:~/nr-lab$ pattern off DBSpamming
DBSpamming enabled=false -> HTTP 200
trainee@nr-lab:~/nr-lab$ date +%H:%M
10:41
after ~5 minutes the TIMESERIES query shows about 6 database calls per booking again
```

Expected output — DBSpamming switched off at the end of the lab

```
SELECT average(databaseCallCount) FROM Transaction
WHERE appName = 'easyTravel-backend' AND request.uri LIKE '%Booking%'
SINCE 30 minutes ago TIMESERIES 1 minute
```

Optional — Make it a key transaction

Open the booking transaction and choose **Make key transaction** (in the … menu). Give it its own Apdex T (for example 0.3 s) so the next regression on checkout is judged against checkout's own standard, not the

backend's average. *If your agent merges all calls into one servlet name, note that a key transaction on that name would not isolate checkout — one more reason to fix naming.*

Check your work

- ☐ You can say which layer, which transaction and which segment held the time.
- ☐ You can name the pattern (N+1) and the evidence: call count per transaction, *Identical Calls* in the trace.
- ☐ DBSpamming is **disabled** again.

Answers. 1 — MongoDB grew; this rules out JVM/CPU problems in the backend's own code and an increase in traffic. 2 — No: each call is fast. The problem is the *number* of calls per transaction.

Lab 1.2 — Follow a failed request end-to-end and pinpoint the failing hop

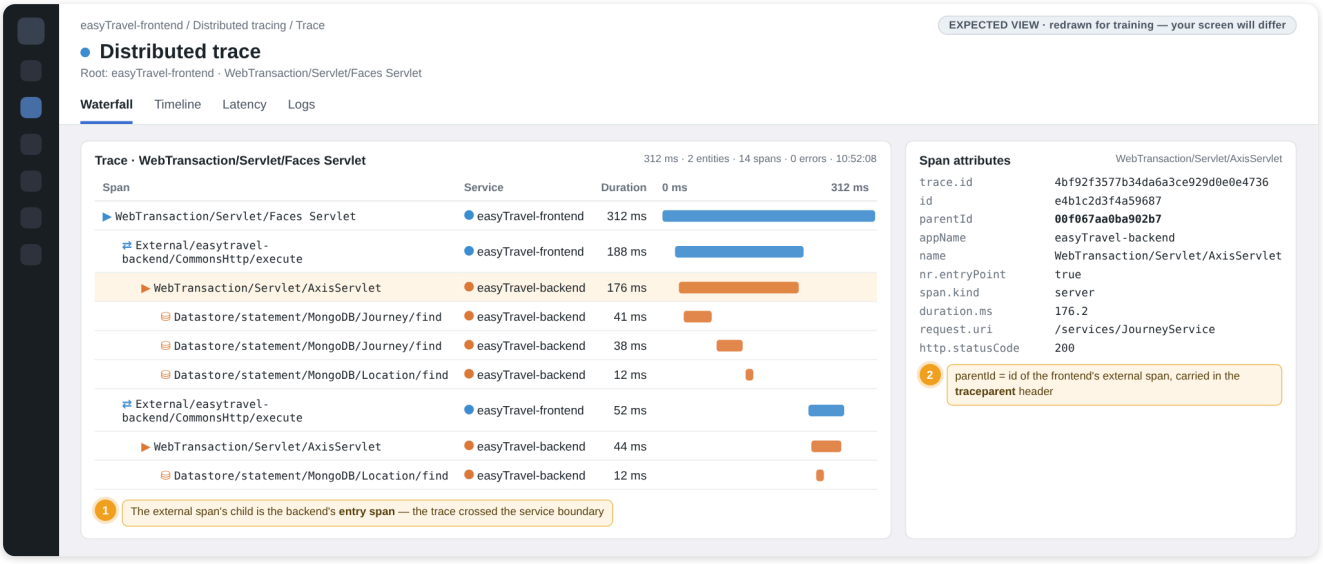
Goal: bookings are failing for customers. Use distributed tracing to follow one failed request from the frontend through the backend to the database, read the error span, confirm the pattern with Span NRQL — and then see what a *broken* trace looks like and fix it.

Time: 50 minutes. **Module:** 2.

Step 1 — Read a healthy trace

Open **APM & Services → easyTravel-frontend → Distributed tracing**. Open a recent trace that includes the backend (the trace list shows the number of entities and spans). In the waterfall:

- The **root span** is the frontend web transaction.
- Under it, an **external** span calls `easytravel-backend:8080`.
- Under that, the **backend** entry span (a different service colour) with its **MongoDB** datastore spans.



Expected view — a healthy distributed trace: frontend, external call, backend, MongoDB spans

Click the backend entry span and open **Attributes**. Note `trace.id`, `parentId`, `appName` and `nr.entryPoint`. The `parentId` is the `id` of the frontend's external span — that link came over the wire in the `traceparent` header.

Step 2 — Break a downstream dependency

```
etdc stop mongodb
date +%H:%M
sleep 40
curl -s -o /dev/null -w "frontend home: HTTP %{http_code} in %{time_total}s\n"
http://localhost/
docker logs --since 2m backend 2>&1 | grep -m3 -iE "mongo|exception"
```

Lab 1.2 · Step 2 · Stop MongoDB — EXPECTED OUTPUT

```
trainee@nr-lab:~/nr-lab$ etdc stop mongodb
[+] Stopping 1/1
 ✔ Container mongodb  Stopped
trainee@nr-lab:~/nr-lab$ date +%H:%M
11:05
trainee@nr-lab:~/nr-lab$ sleep 40
trainee@nr-lab:~/nr-lab$ curl -s -o /dev/null -w "frontend home: HTTP %{http_code} in
                         %{time_total}s\n" http://localhost/
frontend home: HTTP 500 in 30.412s
trainee@nr-lab:~/nr-lab$ docker logs --since 2m backend 2>&1 | grep -m3 -iE "mongo|exception"
com.mongodb.MongoTimeoutException: Timed out after 30000 ms while waiting to connect. Client
    view of cluster
  state is {type=UNKNOWN, servers=[{address=easytravel-mongodb:27017, type=UNKNOWN,
    state=CONNECTING,
  exception={com.mongodb.MongoSocketException: easytravel-mongodb}}]
every backend call that needs the database now waits ~30 s and fails
```

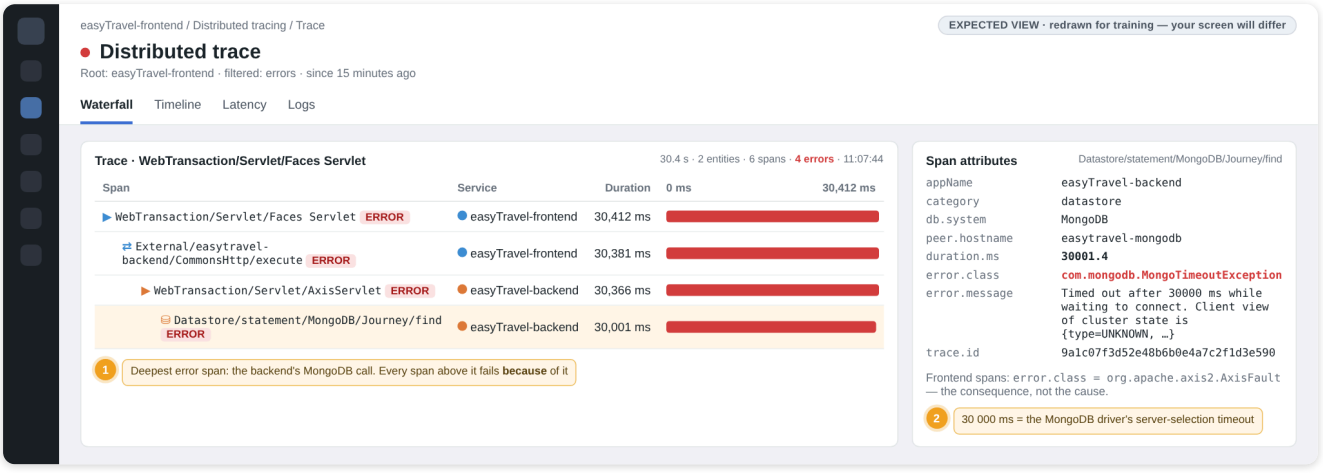
Expected output — MongoDB stopped; backend calls time out after about 30 seconds and log MongoTimeoutException

In the browser, try to search for a journey or book one: the page is slow, then shows an error. The load generator keeps sending traffic, so failures pile up quickly. Wait 3–5 minutes.

Step 3 — Find a failing trace

In **Distributed tracing**, filter on **errors** (the *Errors* toggle or a filter such as `error = true`) for the last 15 minutes. Open the newest trace with errors.

1. Expand the waterfall. Several spans are marked as errors — find the **deepest** one: that is the hop that failed first.
2. Open its **Attributes**: `appName = easyTravel-backend`, the datastore name, `duration.ms` around 30 000, and `error.class` / `error.message` (for example `com.mongodb.MongoTimeoutException` — *Timed out after 30000 ms while waiting to connect*).
3. Compare with the frontend spans above it: their errors are *consequences* (the external call failed or timed out).



Expected view — failed trace: the deepest error span is the backend's MongoDB call, with MongoTimeoutException

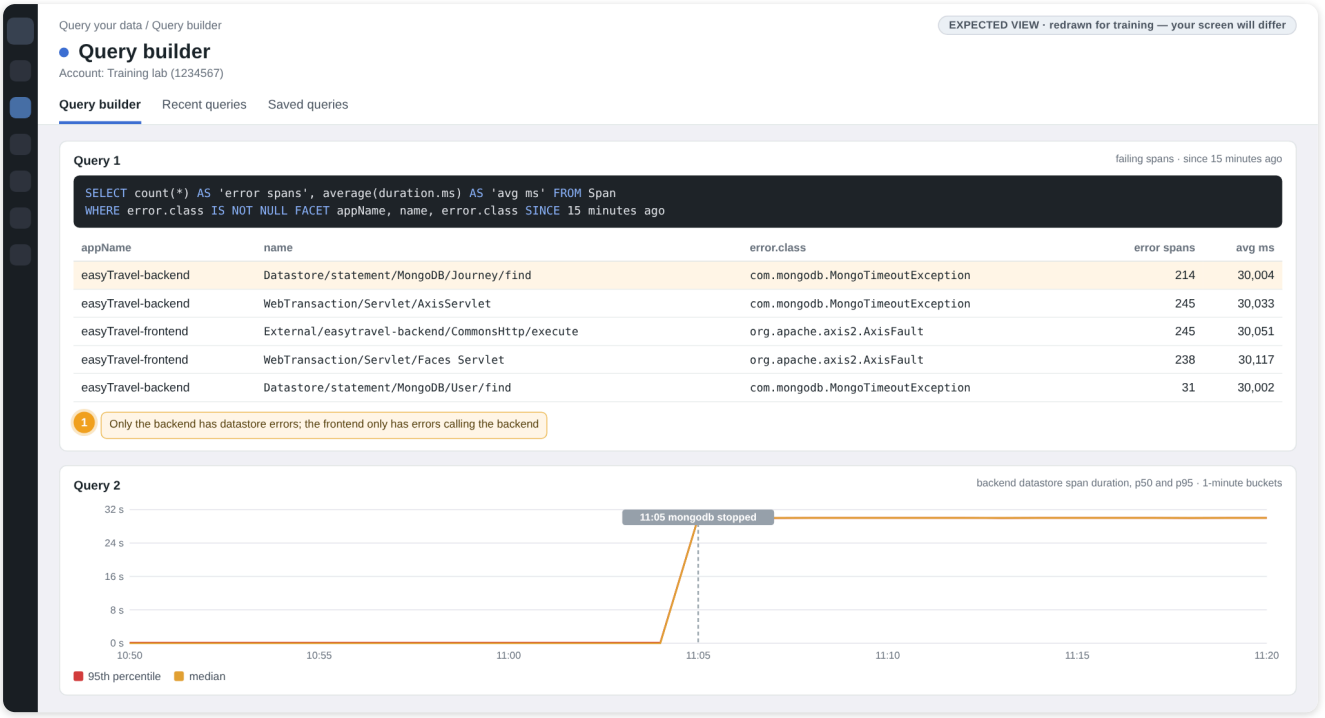
Question 1. The frontend span is red too. Why is the frontend not the failing hop?

Step 4 — Confirm the pattern with Span NRQL

One trace is an anecdote. Prove it across all traffic:

```
SELECT count(*) AS 'error spans', average(duration.ms) AS 'avg ms'
FROM Span WHERE error.class IS NOT NULL
FACET appName, name, error.class SINCE 15 minutes ago
```

```
SELECT percentile(duration.ms, 50, 95), count(*) FROM Span
WHERE appName = 'easyTravel-backend' AND category = 'datastore'
SINCE 30 minutes ago TIMESERIES 1 minute
```



Expected view — Span NRQL: every error starts at a backend datastore span; frontend errors follow

Step 5 — Restore the database

```
etdc start mongodb
sleep 60; curl -s -o /dev/null -w "frontend home: HTTP %{http_code} in %{time_total}s\n"
http://localhost/
```

Re-run the first query in Step 4 with `SINCE 3 minutes ago`: the error spans fall to zero.

Step 6 — See a broken trace

Now break the *trace* instead of the application. Switch distributed tracing off on the backend only:

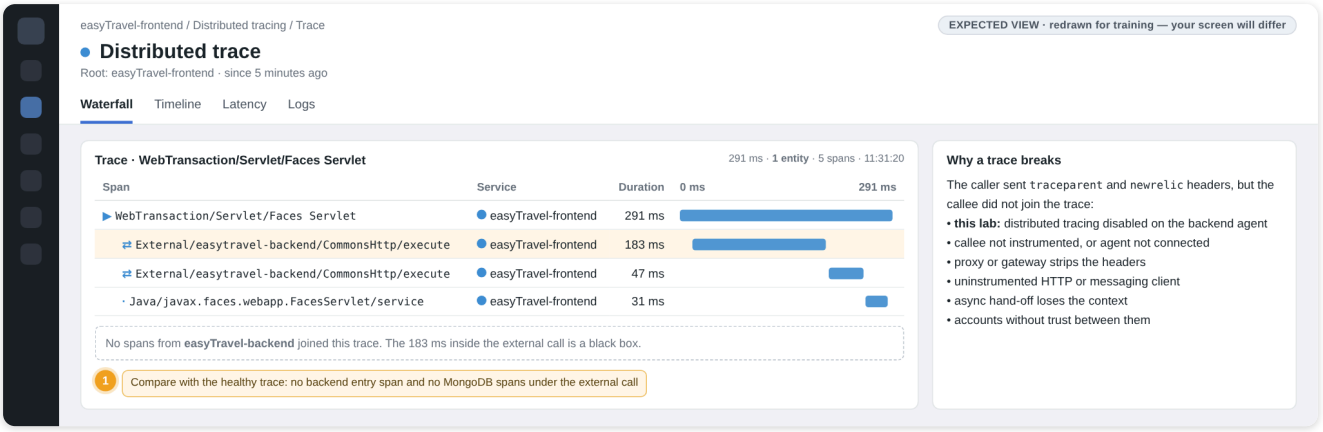
```
sed -i '/^  distributed_tracing:/,/enabled:/ s/enabled: true/enabled: false/' ~/nr-
lab/agent/backend/newrelic.yml
grep -A1 "distributed_tracing:" ~/nr-lab/agent/backend/newrelic.yml
etdc restart backend
```

Lab 1.2 · Step 6 · Distributed tracing off on the backend — EXPECTED OUTPUT

```
trainee@nr-lab:~/nr-lab$ sed -i '/^  distributed_tracing:/,/enabled:/ s/enabled: true/enabled:
                         false/' ~/nr-lab/agent/backend/newrelic.yml
trainee@nr-lab:~/nr-lab$ grep -A1 "distributed_tracing:" ~/nr-lab/agent/backend/newrelic.yml
  distributed_tracing:
    enabled: false
trainee@nr-lab:~/nr-lab$ etdc restart backend
[+] Restarting 1/1
 ✔ Container backend  Started
trainee@nr-lab:~/nr-lab$ sleep 60; grep "Agent v[0-9]"
                         ~/nr-lab/agent/backend/logs/newrelic_agent.log | tail -1
2026-10-26T11:24:52,106+0000 [1 1] com.newrelic INFO: New Relic Agent v8.23.0 is initializing...
```

Expected output — distributed tracing disabled on the backend agent and the backend restarted

Wait 3 minutes and open a new frontend trace. It now **ends at the external span**: the frontend called the backend, but no backend span joins the trace. The waterfall still shows the external call's time — but nothing about what happened inside the backend.



Expected view — a broken trace: the external call has no child spans from the backend

```
-- Backend spans that joined a trace in the last 5 minutes (zero while DT is off)
SELECT count(*) FROM Span WHERE appName = 'easyTravel-backend' SINCE 5 minutes ago
```

Question 2. Name two other causes of a broken trace that you could meet in production.

Step 7 — Fix the trace

```
sed -i '/^  distributed_tracing:/,/enabled:/ s/enabled: false/enabled: true/' ~/nr-
lab/agent/backend/newrelic.yml
etdc restart backend
```

After 3 minutes, open a new trace and confirm the backend and MongoDB spans are back under the external span.

Check your work

- ☐ You found the deepest error span and can name its service, operation and `error.class`.
- ☐ The Span NRQL shows the same failure across all traffic.
- ☐ MongoDB is **running** and distributed tracing is **on** in both `newrelic.yml` files.

Answers. 1 — The frontend's errors are its calls to the backend failing; its own code is fine. The *deepest* error span — the one with no failing children — is where the failure starts. 2 — Any two of: a proxy or gateway that strips `traceparent` / `newrelic` headers; an uninstrumented HTTP client or messaging library; asynchronous work that loses the trace context; a service not instrumented at all; services in accounts without trust between them.

Sampling note. With default head-based sampling the agent keeps about 10 sampled transactions' traces per minute per service, so not every failed request has a trace. Errors still show in the error rate and in `TransactionError`. Infinite Tracing (tail-based) would keep all error traces — it needs a trace observer in your account and is not used in this lab.

Lab 1.3 — Correlate a post-deployment latency spike with traces, logs and the change record

Goal: a new backend release (2.1.0) goes out. Minutes later, logins are slow. Record the deployment in New Relic, then use the deployment marker, the service map, before/after traces and logs in context to prove the release caused the spike — and record the rollback.

Time: 50 minutes. **Module:** 3.

Step 1 — Install the New Relic CLI

```
curl -Ls https://download.newrelic.com/install/newrelic-cli/scripts/install.sh | bash
newrelic version
set -a; source ~/nr-lab/.env; set +a
newrelic profile add --profile lab --region "$NEW_RELIC_REGION" \
  --apiKey "$NEW_RELIC_API_KEY" --accountId "$NEW_RELIC_ACCOUNT_ID" \
  --licenseKey "$NEW_RELIC_LICENSE_KEY"
newrelic profile default --profile lab
newrelic entity search --name easyTravel-backend --domain APM --type APPLICATION | jq -r '.[]
| "\(.name)  \(.guid)"'
```

The profile stores the user key in `~/.newrelic/` — that folder is a secret too.

Step 2 — Record the deployment

In a real pipeline, this command is the last step of the deploy job:

```
newrelic changeTracking create \
  --entitySearch "name = 'easyTravel-backend' AND domain = 'APM' AND type = 'APPLICATION'" \
  --category Deployment --type Basic \
  --version "2.1.0" --commit "9f3c2ab" \
  --description "Login: loyalty points lookup" \
  --user "$USER" \
  --changelog "https://git.example.com/easytravel/releases/2.1.0"
```

Lab 1.3 · Steps 1-2 · New Relic CLI and the deployment record — EXPECTED OUTPUT

```
trainee@nr-lab:~/nr-lab$ newrelic version
newrelic version 0.106.21
trainee@nr-lab:~/nr-lab$ newrelic profile default --profile lab
trainee@nr-lab:~/nr-lab$ newrelic entity search --name easyTravel-backend --domain APM --type
                         APPLICATION | jq -r '.[] | "\(.name)  \(.guid)"'
easyTravel-backend  MTIzNDU2N3xBUE18QVBQTElDQVRJT058NjAxMjM0NTY4
trainee@nr-lab:~/nr-lab$ newrelic changeTracking create --entitySearch "name =
                         'easyTravel-backend' AND domain = 'APM' AND type = 'APPLICATION'"
                         --category Deployment --type Basic --version "2.1.0" --commit
                         "9f3c2ab" --description "Login: loyalty points lookup" --user
                         "$USER" --changelog
                         "https://git.example.com/easytravel/releases/2.1.0"
{
  "category": "Deployment",
  "changeTrackingId": "5b0d4c2e-8f3a-4e61-9c7d-2a1f6e9b3d40",
  "entityGuid": "MTIzNDU2N3xBUE18QVBQTElDQVRJT058NjAxMjM0NTY4",
  "timestamp": 1793023320000,
  "type": "Basic",
  "version": "2.1.0"
}
14:02 - the marker appears on the backend's charts within a minute or two
```

Expected output — New Relic CLI installed; the deployment of easyTravel-backend 2.1.0 recorded

Step 3 — Ship the bad release

`DBSpammingAuthWithAppDeployment` simulates the upgrade of the application and makes every login spam the database:

```
pattern on DBSpammingAuthWithAppDeployment
date +%H:%M
```

Lab 1.3 · Step 3 · Ship the bad release — EXPECTED OUTPUT

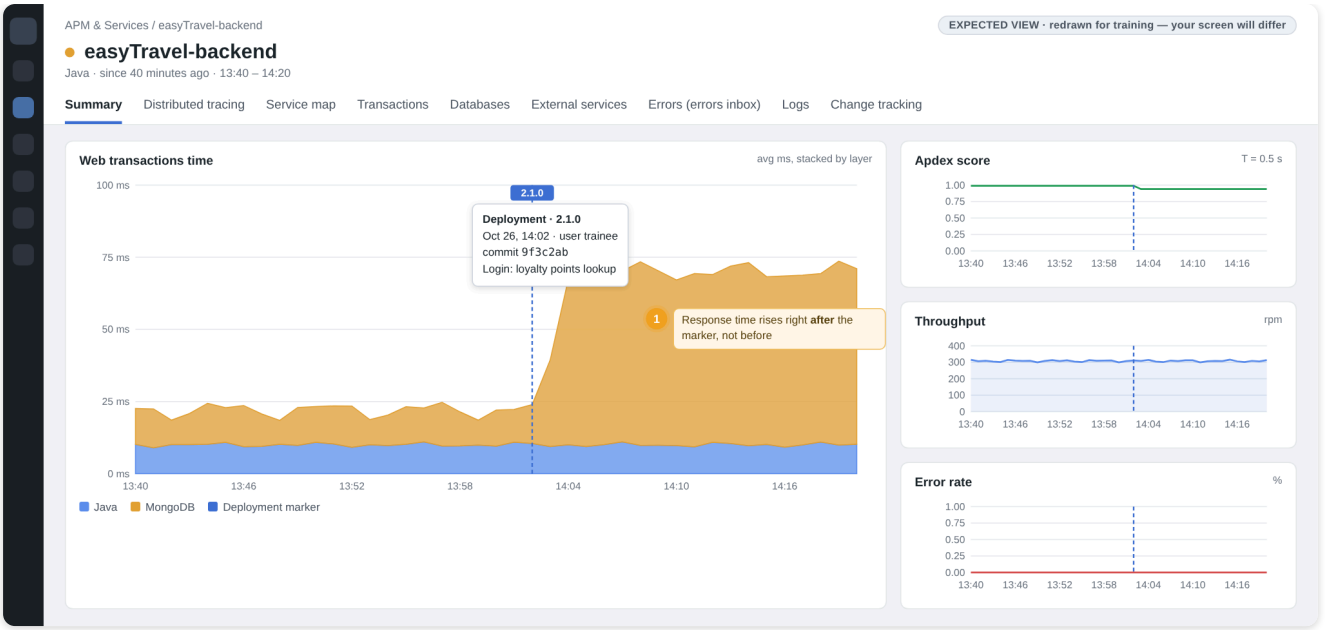
```
trainee@nr-lab:~/nr-lab$ pattern on DBSpammingAuthWithAppDeployment
DBSpammingAuthWithAppDeployment enabled=true -> HTTP 200
trainee@nr-lab:~/nr-lab$ date +%H:%M
14:03
log in and out of easyTravel a few times, then wait 5-10 minutes
```

Expected output — the bad release is live: DBSpammingAuthWithAppDeployment enabled

Log in and out of easyTravel two or three times in the browser. Wait 5–10 minutes.

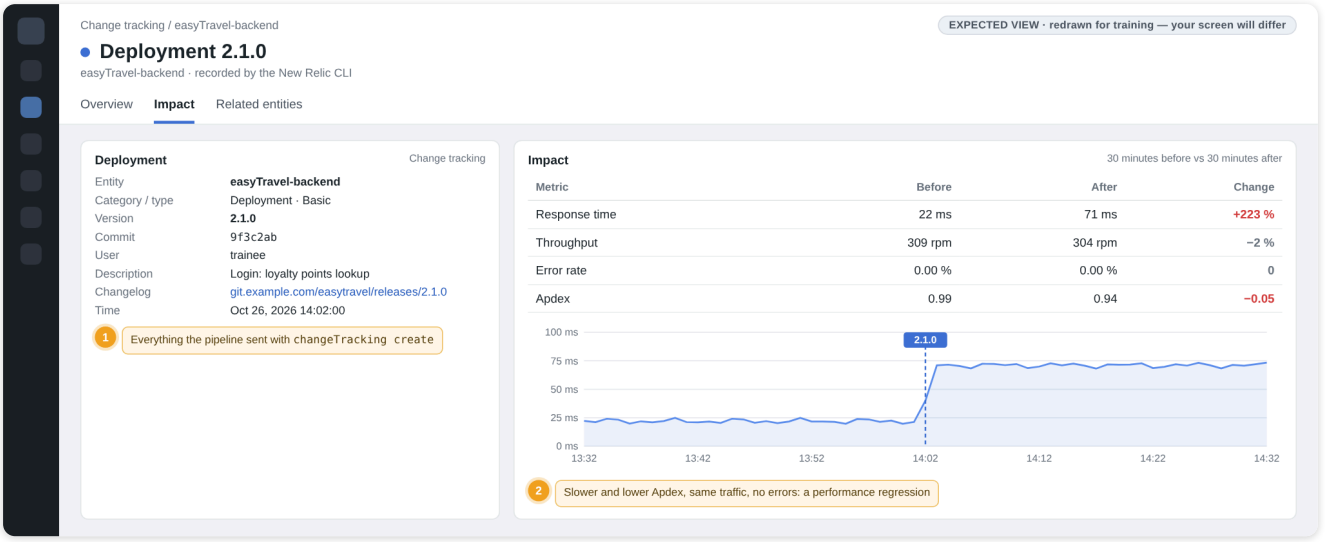
Step 4 — Read the marker

Open **easyTravel-backend → Summary**, last 30 minutes. A **deployment marker** (a vertical line) sits on the charts at the time of Step 2. Hover it: version *2.1.0*, commit, user and description. The web transaction time and the MongoDB share rise **right after** the marker.



Expected view — Summary with the 2.1.0 deployment marker; response time rises right after it

Open the marker (or **Change tracking** in the left navigation) for the **impact** view: response time, throughput, error rate and Apdex before and after the change.



Expected view — change tracking: version 2.1.0, with response time up and Apdex down after the change

Confirm with NRQL:

```
SELECT average(duration), average(databaseCallCount) FROM Transaction
WHERE appName = 'easyTravel-backend' FACET request.uri
SINCE 10 minutes ago COMPARE WITH 40 minutes ago
```

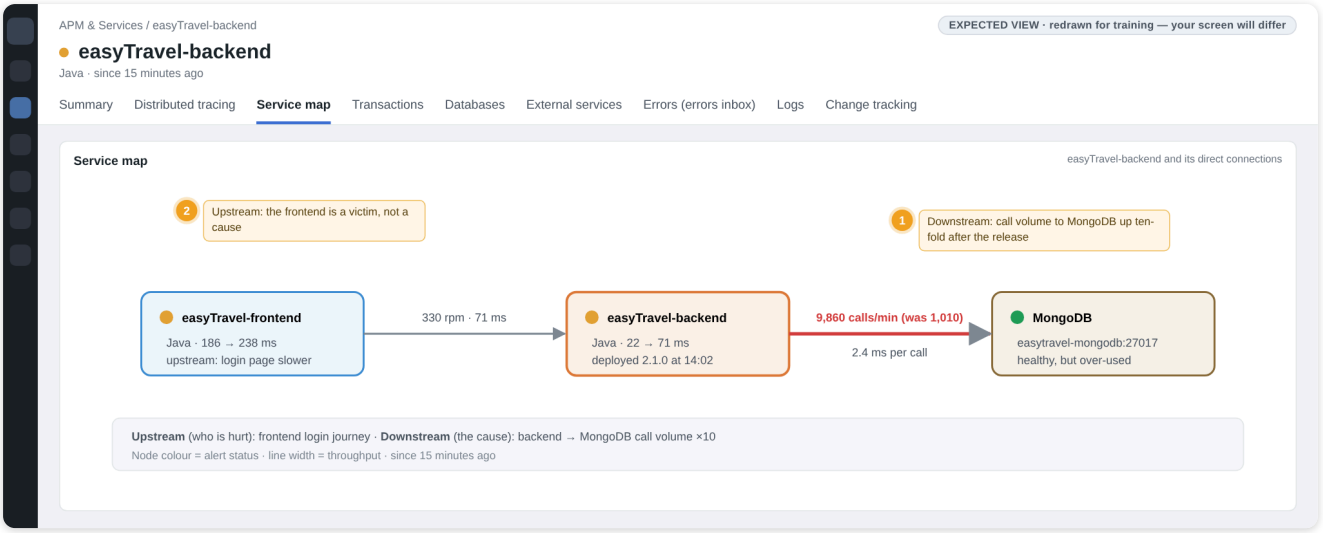
```
SELECT timestamp, version, commit, description, user FROM Deployment
WHERE appName = 'easyTravel-backend' SINCE 1 day ago
```

If the second query returns nothing in your account, use the **Change tracking** view instead — it always lists the recorded changes.

Step 5 — Size the blast radius on the service map

Open **easyTravel-backend → Service map**. The map shows the frontend calling the backend and the backend calling MongoDB.

- **Downstream:** the backend → MongoDB connection shows far more calls per minute than before — the cause.
- **Upstream:** the frontend's calls to the backend are slower — the frontend is a victim. Its login page is the user journey hurt.



Expected view — service map: frontend → backend → MongoDB, with the backend → MongoDB call rate up sharply

Question 1. Who must you tell, and which user journey is affected?

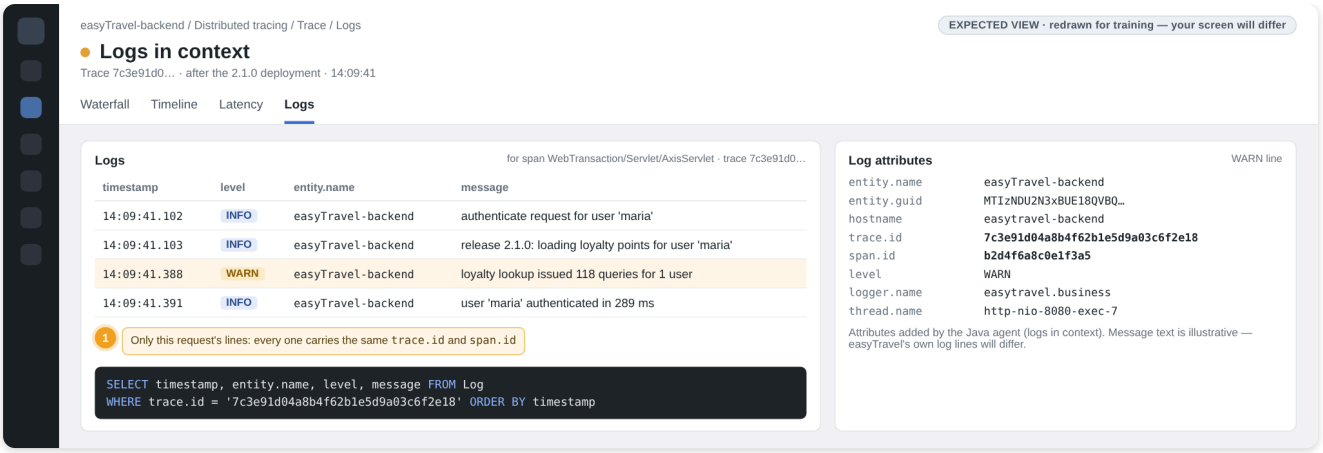
Step 6 — Compare traces before and after

Open **Distributed tracing** and filter on the backend service. Open one trace from **before** the marker and one from **after** it, for the login (authentication) call. After the marker, the login span holds a long run of identical MongoDB calls that the earlier trace does not have.

Step 7 — Read the logs of that request

On the *after* trace, click the backend entry span and open its **Logs** tab: only the log lines written during that request appear, each carrying the trace's `trace.id`. Or query them directly:

```
SELECT timestamp, entity.name, level, message FROM Log
WHERE trace.id = '<paste the trace id>' ORDER BY timestamp
```



Expected view — logs in context: the log lines of one login request, linked by trace.id

> **If no logs appear.** Check that `application_logging.forwarding.enabled` is `true` in `~/nr-lab/agent/backend/newrelic.yml`, look for errors in the agent log (Lab 1.0, Step 7), and run `SELECT count(*) FROM Log WHERE entity.name LIKE 'easyTravel-%' SINCE 30 minutes ago`. easyTravel writes through `java.util.logging`; if your agent version does not forward those lines, install the New Relic infrastructure agent on the VM and forward the containers' logs, then search Logs by time and `hostname` instead. The trace-to-log jump then works only with the trace ID in the log message.

Step 8 — Roll back and record it

```
pattern off DBSpammingAuthWithAppDeployment
newrelic changeTracking create \
  --entitySearch "name = 'easyTravel-backend' AND domain = 'APM' AND type = 'APPLICATION'" \
  --category Deployment --type Basic \
  --version "2.0.9" --description "Rollback of 2.1.0: login DB spamming" --user "$USER"
```

Lab 1.3 · Step 8 · Roll back and record the rollback — EXPECTED OUTPUT

```
trainee@nr-lab:~/nr-lab$ pattern off DBSpammingAuthWithAppDeployment
DBSpammingAuthWithAppDeployment enabled=false -> HTTP 200
trainee@nr-lab:~/nr-lab$ newrelic changeTracking create --entitySearch "name =
                         'easyTravel-backend' AND domain = 'APM' AND type = 'APPLICATION'"
                         --category Deployment --type Basic --version "2.0.9" --description
                         "Rollback of 2.1.0: login DB spamming" --user "$USER"
{
  "category": "Deployment",
  "changeTrackingId": "c71e9a05-2d4b-4f8e-a6b3-9e0d5f7c1a28",
  "entityGuid": "MTIzNDU2N3xBUE18QVBQTElDQVRJT058NjAxMjM0NTY4",
  "timestamp": 1793024580000,
  "type": "Basic",
  "version": "2.0.9"
}
14:23 - a second marker: the spike now sits between the release and the rollback
```

Expected output — the bad release switched off and the rollback recorded as a new change

After 5 minutes the Summary shows two markers — the release and the rollback — with the spike between them. That picture is the incident timeline for the post-incident review.

Check your work

- ☐ Two markers (2.1.0 and 2.0.9) are on the backend Summary.
- ☐ You can show the before/after numbers, the extra MongoDB calls in the *after* trace and the request's log lines.
- ☐ `DBSpammingAuthWithAppDeployment` is **disabled**.

Answer. 1 — The login journey on the customer site (frontend) is affected; tell the owners of the frontend and the backend, and whoever approved release 2.1.0. MongoDB itself is healthy — it is being over-used.

Appendix — Reset, troubleshooting and end-of-day checklist

Reset everything

```
for p in DBSpamming DBSpammingAuthWithAppDeployment; do pattern off $p; done
etdc start mongodb
grep -A1 "distributed_tracing:" ~/nr-lab/agent/*/newrelic.yml
etdc ps --format "table {{.Name}}\t{{.Status}}"
```

To remove the lab completely: `etdc down -v`, then delete `~/nr-lab` and `~/.newrelic`, and revoke the user key in New Relic (*API keys*).

Troubleshooting

Symptom	Check	Fix
A service never appears in APM	`grep -i -e error -e license ~/nr-lab/agent/<tier>/logs/newrelic_agent.log`	Wrong or missing license key: fix `~/nr-lab/.env`, then `etdc up -d backend frontend`
No agent log file at all	`etdc exec backend sh -c 'echo $CATALINA_OPTS'`	The override file was not used — always start with the `etdc` alias
Agent log: *unable to connect*, timeouts	`curl -sI https://collector.newrelic.com` from the VM	Allow outbound HTTPS to New Relic (EU accounts: `collector.eu.newrelic.com`)
Backend container restarts	`docker logs --tail 20 backend`	Usually *Waiting for the easyTravel database*: start `mongodb` first
Distributed traces show only one service	`grep -A1 distributed_tracing ~/nr-lab/agent/*/newrelic.yml`	DT must be `true` on **both** tiers; restart the tier you changed
`curl` to 8091 returns nothing	`etdc ps backend`	The backend is still starting (Tomcat takes about a minute)
No `Log` events	`application_logging` block in `newrelic.yml`	See the note in Lab 1.3, Step 7
The VM is slow	`docker stats --no-stream`	The headless load generator uses Chrome; stop it between labs with `etdc stop headless-customer-loadgen`

End-of-day checklist

- ☐ All problem patterns are disabled and MongoDB is running.
- ☐ Distributed tracing is `true` in both `newrelic.yml` files.
- ☐ Your NRQL queries from the four labs are saved (a dashboard or a notebook works well).

- ☐ `.env` and `~/.newrelic/` are not in any Git repository.
- ☐ If the VM is shared or will be deleted: user key revoked.

Sources

- Dynatrace — easyTravel-Docker on GitHub and Available easyTravel problem patterns
- New Relic docs — Java agent: configure logs in context, How New Relic distributed tracing works, Introduction to transaction traces, Service maps, Change tracking